
◆ What is Routing in ASP.NET Core MVC?

Routing is the mechanism that maps an incoming HTTP request (URL) to a specific **Controller** → **Action Method**.

👉 In simple terms:

Routing decides “**which code should handle this URL request?**”

◆ Evolution of Routing (Important Context)

In modern ASP.NET Core (including **.NET 10**), routing is based on:

- **Endpoint Routing (introduced in ASP.NET Core 3.0)**
- Fully integrated into middleware pipeline

👉 Key idea:

Routing is no longer MVC-specific — it’s a **central system** used by MVC, Razor Pages, APIs, etc.

◆ How Routing Works Internally

Request Flow:

1. Client sends request
 2. `https://example.com/Product/Details/5`
 3. Routing middleware matches URL pattern
 4. It extracts values:
 - Controller = Product
 - Action = Details
 - Id = 5
 5. Executes:
 6. `ProductController → Details(int id)`
-

◆ Types of Routing in ASP.NET Core MVC

1 Conventional Routing (Default Routing)

Defined centrally in **Program.cs**

Example:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Explanation:

Segment Meaning

controller Controller name

action Action method

id? Optional parameter

👉 Default values:

- Controller → Home
- Action → Index

Example URLs:

URL	Result
/	HomeController → Index
/Home/About	HomeController → About
/Product/Details/10	ProductController → Details(10)

2 Attribute Routing

Defined directly on controllers and actions using attributes.

Example:

```
[Route("product")]
```

```
public class ProductController : Controller
{
    [Route("list")]
    public IActionResult List()
    {
        return View();
    }

    [Route("details/{id}")]
    public IActionResult Details(int id)
    {
        return View();
    }
}
```

URLs:

- /product/list
- /product/details/5

3 Mixed Routing

You can use both:

- Conventional + Attribute routing together

👉 Common in real-world apps

◆ Route Parameters

Example:

pattern: "{controller}/{action}/{id?}"

- {id} → required
- {id?} → optional

Multiple Parameters:

```
[Route("order/{orderId}/item/{itemId}")]
```

◆ Default Values in Routing

pattern: "{controller=Home}/{action=Index}/{id?}"

If URL is /, it becomes:

HomeController → Index()

◆ Route Constraints

Used to restrict parameter values.

Example:

```
[Route("product/{id:int}")]
```

👉 Only matches integers

Other Constraints:

- int
 - bool
 - datetime
 - guid
 - minlength
 - maxlength
-

◆ Routing Configuration in .NET 10 (Program.cs)

In .NET 10, minimal hosting model continues.

Basic Setup:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

var app = builder.Build();

app.UseRouting();

app.UseEndpoints(endpoints =>
{
    endpoints.MapControllerRoute(
        name: "default",
        pattern: "{controller=Home}/{action=Index}/{id?}");
});

app.Run();
```

👉 OR simplified modern syntax:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

◆ URL Generation (Reverse Routing)

Routing is also used to generate URLs.

Example:

```
@Url.Action("Details", "Product", new { id = 5 })
```

👉 Output:

/Product/Details/5

◆ Route Order & Priority

- Attribute routes → Higher priority
 - Conventional routes → Lower priority
-

◆ Key Concepts Summary

Concept	Description
Endpoint Routing	Central routing system
Conventional Routing	Defined in Program.cs
Attribute Routing	Defined using attributes
Route Parameters	Dynamic values in URL
Constraints	Validate parameters
Default Values	Fallback values
URL Generation	Reverse routing

◆ Final Insight

Routing is the **entry point of your MVC application**.

If routing fails → your application cannot respond correctly.
